Arrays

12 October 2026 19:22

Arrays are contiguous allocation of memory (one after the other with no gaps)
- → Static (initialized with fixed size)
- → Dynamic (has no fixed size)

In python you can implement using
1) python lists
2) array module (for typed array/seq)
3) by numpy array (for other numerical computation)

For DSA use lists (dynamic arrays)

In python to define static arrays use array module or numpy.
For dynamic → lists!

Q. How does python allow dynamic array unlike C++, Java, C? How is it allocated memory?
python keeps extra space and once size == capacity it auto allocates.

How arrays work inside a computer?

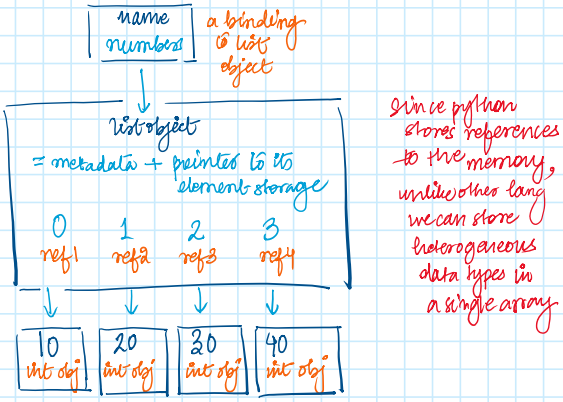
Steps: We define a list
numbers = [10, 20, 30, 40]

↓

Step2: python runtime executes the code
Create objects and bind names to them

↓

Step3: memory hold the runtime data
the list, its storage slot, & the int obj



Since python stores references to the memory, unlike other lang we can store heterogeneous data types in a single array.

Traditionally, in a simplified programming model:

RAM
- → Stack → function calls = execution state, local variable execution resumes
- → Heap → objects + dynamically allocated data whose lifetimes are beyond single func call.

list lives here

* they are concepts, not physical parts

Python list operations:

1) Access by index
2) Update by index
3) Search for an arbitrary value
4) Append at the end
5) insert or delete near the beginning
6) iterate through all elements

O(1) → Address(i) = Address of 1st element + i * bytes element takes
since cpython can directly find element using base address directly, it only takes O(1)

O(1)
O(1)
O(n)
Amortized O(1) → Amortized as the avg of operations is O(1) why? as for dynamic array python allocates new memory by a growth factor 1.125
0 → 4 → 8 → 16 → 24 → 32 .....
O(n) → since all the elements shifts to higher indices, so potentially coping n references takes O(n)
O(n)

Algorithms

Q.1 find largest element in array:

```
def find_largest_element(arr: list[int]) -> int:
    largest_element = arr[0]          # O(1) space
    for item in arr:                  # O(n) time
        if item > largest_element:    # loop invariant = remains true at particular point in every iteration
            largest_element = item
    return largest_element
```

Problem solving template:
1) Understand the problem → what must be returned? what are the constraints?
2) work through a small example - manually
3) Develop a brute force approach
4) identify repeated work or unnecessary operations → helps optimize it
5) prove correctness - why it works
6) analyze complexity

largest_element = item iteration

return largest_element

4) identify repeated work or unnecessary operations
→ helps optimize it
5) prove correctness – why it works
6) analyze complexity

Q.2 Given array, determine elements are sorted in non-decreasing order (ascending)
You cannot sort the array.

Framework:

1) understand the problem:
Return: True or False
Constraints: Cannot sort the array

2) work through a small example.
[2, 4, 7, 6, 9]
From human's perspective → it is not sorted → False
From coding →
For each element check if next is smaller,
if yes False
else True

3) Brute force algorithm

```
def check_array_is_sorted(arr: list[int])
    -> bool:
    if len(arr) == 0:
        raise ValueError("Array is empty")

    for i in range(0, len(arr) - 1):
        if arr[i] > arr[i+1]:
            return False

    return true
```

time O(n)
O(1) space

4) How to fail this?
→ Give empty array
immunity to duplicates

[1, 2, 2, 4, 5] → 1>2 × 2>2 × 4>5 ×
2>4 × 2>4 ×

Q.3 Find the second largest distinct value without sorting the array

1) understand the question
Return → second smallest element or None
Constraint → array cannot be sorted

Examples: [12, 35, 1, 10, 34, 1] → 34
[5, 5, 4] → 4
[7, 7, 7] → None
[-2, -1, -2] → -2
[8] → None

2) manually try an example
[7, 8, 2, 5]
1) find the largest element → 8
then the element that couldn't be the
largest or the immediate smallest is the one.

Passed [12, 35, 1, 10, 34, 1]
largest = 12
2nd = -something

for loop:

Pass 1 35 > 12
2nd = 12
largest = 35
35 > 2nd and 35 < 35 ×

Pass 2 1 > 35 ×
1 > 12 ×

Pass 3 10 > 35 ×
10 > 12 ×

Pass 4 34 > 35 ×
34 > 12 ✓ and 34 < 35
second largest = 34

Pass 5 1 > 35 ×
1 > 34 ×

2) Brute force:

```
def second_smallest_element(arr: list[int]) -> int | None:
    if len(arr) < 2:
        None

    largest_element = arr[0]
    second_largest_element = None   no float(-inf)

    for i in range(1, len(arr)):
        if arr[i] > largest_element:
            second_largest_element = largest_element
            largest_element = arr[i]

        if arr[i] > second_largest_element and arr[i] < largest_element:
            second_largest_element = arr[i]

    return second_largest_element
```

if arr[i] < largest_element
(if second_largest_element is None
and arr[i] > second_largest_element)
☆ so that in first pass there is only one check.

[7, 7, 7]
7 > 7 × 7 > 7 ×
7 < 7 × 7 < 7 ×
largest = 7
2nd = float(-inf)